```
fs = require 'fs'
{Post} = require './models/post'

load = (dir) ->
  fs.readdir dir, (err, files) ->
    for file in files when /.*[.]md$/.test file
      fs.readFile "#{dir}/#{file}", 'utf-8', (err, data) ->
        [title, content...] = data.split '\n'
        new Post title, content.join '\n'

exports.load = load
```

Read all the files with extension .md in the supplied directory and create Post objects from each one.

The next listing shows the `Controller` class. This controller can handle incoming HTTP requests and respond as desired. It invokes a method on a particular view in order to generate the body of the response.

Listing 12.4. controllers/controller.coffee

```
class Controller
  routes = {}

  @route = (path, method) ->
    routes[path] = method
    constructor: (server) ->
      server.on 'request', (req, res) =>
        path = require('url').parse(request.url).pathname
        handlers = []
        for route, handler of routes
          if new RegExp("^#{route}$").test(path)
            handlers.push
              handler: handler
              matches: path.match(new RegExp("^#{route}$"))
        method = handlers[0]?.handler || 'default'
        res.end @[method](req,res,handlers[0]?.matches.slice(1)...)

    render: (view) ->
      @response.writeHead 200, 'Content-Type': 'text/html'
      @response.end view.render()

    default: (@request, @response) ->
      @render render: -> 'unknown'

  exports.Controller = Controller
```

The controller constructor attaches its handler to the provided server. This handler looks at incoming requests and decides how the response is created.

The render method writes headers and then the response body by rendering the provided view.

A default request handler for when no handler is found to render a response.

In the next listing you see the `Blog` class that inherits from the `Controller` class. This inheritance is a by-product of your chosen design and isn't necessary when you have only a single blog controller. But it does help to demonstrate real-world use of modules. To have `Blog` inherit from `Controller`, the blog module needs to first load the controller module.

Listing 12.5. controllers/blog.coffee